Git Hosting Service DBMS

Group G-01

Members:

BCSF25M005	Muhammad Subhani
BCSF25M013	Abdur Rehman Imran

Project Description

Title: Git Hosting Service DBMS: a database for hosting and collaborating on code.

Problem statement: Sites like Codeberg, GitHub feel simple to use, but behind the scenes there is a whole mesh in which everything is connected. People belong to organizations and teams, teams get access to repositories, and repositories hold issues, pull requests, reviews, comments and notifications. A simple database cannot handle all these without taking proper care of all the links between data. Our project designs a clean relational database for a Git hosting service that controls who can do what, keeps a clear history of collaboration, and measure other metrics.

Stakeholders

Our main stakeholders are:

- **Developers and contributors:** write code, open issues and pull requests, and join the discussion.
- **Maintainers and reviewers:** review pull requests and either approve them or ask for changes.
- **Organization and team admins:** decide who is on which team and who can access which repository.
- **Project managers:** keep track of open issues and how quickly reviews happen.

Importance

Git hosting is how most software teams work together. What makes it interesting is how the pieces connect: the **relationships**.

- **Relationship-heavy by nature:** users and teams, teams and repositories, and pull requests and reviewers all link many-to-many, and comment replies point back to other comments. Getting this right takes careful keys and constraints.
- **Flow:** the design follows real workflows, like a pull request moving from open to reviewed to merged, with notifications sent along the way.
- **Rich, meaningful queries:** we can answer questions how a team is collaborating and working together.
- **Original and hand-designed:** we design every table, constraint and query ourselves, based on how code hosting really works.

Sample Questions

1. Which pull requests in a repository are still open, who wrote them, and which reviewers haven't responded yet?
2. Who can write to a given repository, either directly or through a team that has access?
3. For each user, how many unread notifications do they have? grouped by type (mention, review request, reply, issue assignment)

Entity List

#	*Entity*	*Purpose and key relationships*
1	User	Account holder; owns repositories, authors issues, pull requests and comments
2	Organization	Owns repositories and teams
3	Team	Group of users inside an organization
4	Repository	Owned by a user or organization; has visibility; self-references its parent when forked
5	Issue	Task or bug in a repository; has author, assignees and optional milestone
6	Pull Request	Proposed merge from a source branch to a target branch; has author and state
7	Review	Verdict on a pull request (approved, changes requested, commented)
8	Comment	Message on an issue, pull request or review; self-references for replies
9	Label	Tag applied to issues and pull requests (many-to-many)

10	Release	A published version of a repository, tied to a Git tag; has a title, notes and the user who published it

AI-Built Application Track

Decision: We do not plan to use the optional AI-built application track. We will keep our efforts on the database itself: schema design, constraints, queries and testing.